# Guion para leer y exponer

Qolqe + Robot Framework | Grupo 4 | 10 minutos

> Antes de exponer (no leer): iniciar Qolqe con la terminal 1 del README; dejar listas la terminal 2, el repositorio en Codex y la guía docs/demo-robot-ia.md. Los recuadros verdes indican acciones que se muestran en vivo.

## DIAPOSITIVA 1 | Portada | 0:00-0:20

Hola. En este trabajo automatizamos un recorrido completo del módulo de solicitudes de dinero de Qolqe. Diego agrega a Bruno como contacto, le pide 25 soles y Bruno acepta. Lo hicimos con Robot Framework y el patrón Page Object. Al final veremos la prueba ejecutándose y una revisión con inteligencia artificial.

## DIAPOSITIVA 2 | ¿Por qué Robot Framework? | 0:20-1:00

Robot Framework es una herramienta para automatizar pruebas usando palabras clave fáciles de leer. En lugar de repetir manualmente todos los clics, describimos el recorrido una vez y dejamos que la prueba lo ejecute y compruebe el resultado. En Qolqe no basta con pulsar “Enviar solicitud”: también queremos saber si quedó Pendiente, si después fue Aceptada y si el dinero realmente cambió de cuenta. Ese es el problema que buscamos resolver.

## DIAPOSITIVA 3 | Cómo ha evolucionado | 1:00-1:40

Robot Framework comenzó muy ligado a las pruebas de aceptación: casos expresados con palabras clave, bibliotecas para interactuar con sistemas y reportes claros. Luego amplió su uso a la automatización de tareas repetitivas, lo que se conoce como RPA. Para aplicaciones web actuales podemos usar Browser Library, que se apoya en Playwright. En este proyecto esa combinación controla un navegador Chromium real, igual que el que usaría una persona.

## DIAPOSITIVA 4 | Arquitectura y componentes | 1:40-2:35

La arquitectura se puede leer de izquierda a derecha. Primero está nuestro archivo de prueba, terminado en .robot, donde escribimos las acciones de negocio. Luego están los recursos Page Object: ahí viven los clics, los campos, las esperas y las comprobaciones de cada pantalla. El motor de Robot ejecuta esas palabras clave. Browser Library se comunica con Playwright, Playwright controla Chromium y el navegador interactúa con Qolqe. Esta separación ayuda porque el caso no necesita saber dónde está cada botón: solo expresa qué resultado esperamos. Al terminar, Robot produce output.xml, log.html y report.html. Así tenemos tanto el resultado resumido como el detalle de cada paso.

## DIAPOSITIVA 5 | Patrón Page Object | 2:35-3:40

El patrón que aplicamos es Page Object. En la izquierda vemos que el caso dice qué debe ocurrir: enviar la solicitud, aceptarla y comprobar estado y saldos. A la derecha vemos cómo se hace una de esas acciones: el recurso SolicitudesPage usa un selector para escribir el monto y otro para pulsar el botón. Cuando Bruno acepta, otra palabra clave reúne el clic de aceptación, el cuadro de confirmación y la espera por el nuevo estado. Elegimos este patrón porque el recorrido pasa por varias pantallas y por dos usuarios. Si Qolqe cambia un selector, lo corregimos en el recurso de esa pantalla; el caso principal sigue siendo legible y no tenemos que reescribir toda la historia.

## DIAPOSITIVA 6 | Diego inicia el recorrido | 3:40-4:45

El happy path empieza con Diego. Entra a Qolqe y agrega a Bruno como contacto. La prueba no se queda solo con el clic en “Agregar”: busca el alias Bruno Demo y comprueba que aparece en la lista. Después Diego le solicita 25 soles con el motivo “Devolución demo G4”. El monto y el motivo están definidos en la suite, de modo que la prueba identifica esta solicitud concreta y no cualquier fila que esté en la pantalla. La segunda captura muestra el resultado que esperamos en ese momento: una solicitud enviada por 25 soles y en estado Pendiente. Estas son las primeras validaciones del flujo.

## DIAPOSITIVA 7 | Bruno acepta y Qolqe transfiere | 4:45-5:55

Ahora cambiamos de usuario. Bruno entra, encuentra la solicitud recibida y confirma que quiere transferir los 25 soles a Diego. Después de aceptar, la prueba verifica que el estado diga Aceptada y que aparezca un código de operación. También comprueba el efecto en los saldos: Bruno empieza con 800 soles y termina con 775; Diego empieza con 2,500 y termina con 2,525. En este caso la comisión es cero. La etiqueta Aceptada por sí sola no sería suficiente: por eso comprobamos también la operación y el cambio observable de dinero. Finalmente, volvemos a la cuenta de Diego y comprobamos que él también ve la solicitud como Aceptada.

## DIAPOSITIVA 8 | Demo del flujo ejecutándose | 5:55-7:30

Pasemos a la ejecución. Voy a lanzar Robot con el navegador visible. El caso prepara los datos de una instancia local de pruebas, entra primero como Diego y después como Bruno, y hace las comprobaciones que acabamos de revisar.

> No leer: con Qolqe ya iniciado, ejecutar en otra terminal: python -m robot --variable HEADLESS:False --outputdir reportes tests/e2e/g4_solicitud_happy_path.robot. Luego abrir reportes/report.html y expandir un paso en reportes/log.html.

Aquí vemos el resultado: una prueba ejecutada y aprobada. En el log podemos abrir los pasos y ver que Robot comparó los estados, el código de operación y los saldos. El reporte queda guardado, de modo que la ejecución se puede revisar y repetir; no dependemos solo de una captura de pantalla.

## DIAPOSITIVA 9 | IA aplicada en vivo | 7:30-9:10

Ahora voy a usar IA como apoyo para revisar nuestra prueba de Robot Framework. Le daré a Codex el resultado real, el archivo de la prueba y los criterios de aceptación. Le pediré que diga qué se comprobó, qué falta y qué palabra clave pequeña propondría para cubrirlo. La IA no ejecuta ni valida por sí sola esta prueba: nos ayuda a detectar una posible brecha.

> No leer: abrir docs/demo-robot-ia.md, copiar el prompt completo en Codex y mostrar la respuesta. Usar reportes/output.xml, tests/e2e/g4_solicitud_happy_path.robot y docs/requisitos.md como contexto.

Comparemos la respuesta con los requisitos. La brecha que esperamos detectar es el SMS simulado: el caso actual verifica estado, operación y saldos, pero todavía no comprueba la notificación a Bruno ni la que recibe Diego después de la aceptación. Si la IA propone un keyword para consultar notificaciones, eso sigue siendo una propuesta. Para decir que está cubierto, tendríamos que incorporarlo y ejecutar la prueba otra vez. Si Codex pasa por alto el requisito, se lo señalamos: la IA tampoco reemplaza nuestro criterio.

## DIAPOSITIVA 10 | Hallazgos, límites y entrega | 9:10-10:00

Para cerrar, encontramos una dificultad real: justo después de aceptar, la pantalla todavía podía mostrar Pendiente durante un instante. Lo resolvimos esperando a que el estado Aceptada fuera visible antes de comprobarlo. También hay límites: instalar el navegador y sus dependencias toma tiempo, las pruebas por interfaz son más lentas que las de API y los selectores requieren mantenimiento. Por eso no usaríamos este enfoque para cada regla pequeña ni para servicios externos que no controlamos. El repositorio incluye el código, el README para ejecutar la demo y esta presentación. Gracias.